Robotics and Computer Integrated Manufacturing 68 (2021) 102074

Contents lists available at ScienceDirect

Robotics and Computer Integrated Manufacturing

journal homepage: www.elsevier.com/locate/rcim

Unified modeling for digital twin of a knowledge-based system design

Haoqi Wang, Hao Li*, Xiaoyu Wen, Guofu Luo

Henan Key Laboratory of Intelligent Manufacturing of Mechanical Equipment, Zhengzhou University of Light Industry, 450002, PR China

ARTICLE INFO

Keywords:
Model-based systems engineering
Digital twin
System design
Unified modeling
Systems modeling language

ABSTRACT

While Model-Based Systems Engineering (MBSE) improves the ambiguity problem of the conventional document-based way, it brings management complexity. Faced with the complexity, one of the core issues that companies care about is how to effectively evaluate, predict, and manage it in the early system design stage. The inaccuracy of contemporary complexity measurement approaches still exits due to the inconsistency between the actual design process in physical space and the theoretical simulation in virtual space. Digital Twin (DT) provides a promising way to alleviate the problem by bridging the physical space and virtual space. Aiming to integrate DT with MBSE for the system design complexity analysis and prediction, based on previous work, an integration framework named System Design Digital Twin in 5 Dimensions was introduced from a knowledge perspective. The framework provides services for design complexity measurement, effort estimation, and change propagation prediction. Then, to represent the system design digital twin in a unified way, a modeling profile is constructed through SysML stereotypes. The modeling profile includes System design digital model in virtual space profile, system services profile, relationships profile and digital twin data profile. Finally, the system design of a cube-satellite space mission demonstrates the proposed unfiled modeling approach.

1. Introduction

The development of a complex product such as a vehicle, an airplane, or a satellite has evolved to employ a system engineering methodology. In the conventional Document-Based Systems Engineering (DBSE), designers share information and design intent through a set of documents, such as requirements spreadsheets, interface definition diagrams, and design specifications. However, DBSE shows limitations, where the growth in system complexity has increased the risk to unacceptable levels. Ambiguity tends to arise when designers scatter information throughout a disjointed set of documents [1]. As a result, designers from different disciplines have to spend a lot of time understanding the design intent and avoiding ambiguity.

Model-Based Systems Engineering (MBSE) improves on DBSE by formalizing the application of modeling to support requirements analysis, function analysis, and design synthesis [2]. Use of MBSE follows the traditional "Vee" model, while this paper focuses on system decomposition and definition, that is, the left part of the "Vee" model, which is the system design (SD) [3]. Researchers have proposed promising MBSE methodologies like Harmony Systems Engineering [4], Object Process Methodology (OPM) [5], Vitech MBSE [6], and System Modeling Methodology [7]. Although using unified languages, such as SysML [8] and OPL [5], the methodologies can specify the system model iteratively. It is still time-consuming for designers to implement them. In other words, MBSE brings complexity management issues while improving the ambiguity problem [2]. Because the existence of factors like uncertain design information, the coupling of design functions, and changes in design status has increased the complexity, controlling the progress of the MBSE process becomes difficult, and product development time exceeds expectations.

Faced with this complexity, one of the core issues that companies care about is how to effectively evaluate, predict, and manage it in the early SD stage. For this reason, different researchers have proposed various methods, such as information entropy theory [9], computational complexity theory [10], and axiomatic design [11], and Bashir-Zhang-Thomson (BZT) method [12]. However, these methods still have inaccurate problems. For example, the information entropy theory is suitable for unpredictable system behavior, but the ability to accurately display the complexity changes is insufficient [13]. The information axiom in the axiomatic design can evaluate design complexity through achieving functional probability, but it rarely focuses on the design process [3]. While the BZT method can simulate the academic workload of design under different scenarios, it lacks consideration of the impact of actual engineering changes on complexity [14]. The main reason for such drawbacks is that the actual design process in physical space is inconsistent with the theoretical simulation in virtual space.

* Corresponding author.
E-mail addresses: lihao@zzuli.edu.cn (H. Li), wenxiaoyu@zzuli.edu.cn (X. Wen), luoguofu@zzuli.edu.cn (G. Luo).

https://doi.org/10.1016/j.rcim.2020.102074
Received 12 March 2020; Received in revised form 2 September 2020; Accepted 21 September 2020
Available online 04 October 2020
0736-5845/ © 2020 Elsevier Ltd. All rights reserved.

The emergence of Digital Twin (DT) provides an effective way to solve the problem of inaccurate complexity prediction by bridging the physical space and the virtual space [15–17]. The concept of DT can date back to Grieves's presentation about Product Lifecycle Management (PLM) in 2003 [18], and it becomes more mature and specific until 2011. Gartner continuously lists DT as the current top strategic technology development direction in 2016, 2017, and 2018. DT is defined as a digital replica of a system and maintained as a virtual equivalent throughout the physical product's lifespan, which ensures consistency between the ideal model and the actual state by providing smart and real-time services [19–21].

Significant researches have proven the significance of the integration of DT and MBSE. For example, Bachelor et al. present how DT changes how the MBSE process is managed, overcoming the issues associated with federated IT infrastructures, and tools integration [22]. Thomas et al. propose a holistic system lifecycle engineering approach that can close the loop between system architectures and DTs [23]. Erikstad et al. propose a set of reusable design patterns for DTs, which can be applied to different classes of SD and operation problems [24]. Schluse et al. describe an experimental DT for the gap between the first systems engineering steps, and the various algorithms simulation technology can offer [25].

However, seldom researches focus on integrating DT with MBSE for the SD complexity analysis and prediction. As SD can be viewed as the supply and demand of knowledge between designers and product functions, from a knowledge perspective, this paper firstly introduces a framework for integrating DT with MBSE called System Design Digital Twin in 5 Dimensions (SDDT5D). The frame provides services for design complexity measurement, effort estimation, and change propagation prediction. Then, to represent the system design digital twin in a unified way, a modeling profile is constructed through SysML stereotypes. The system model reduces ambiguity through unifying system requirements, functions, and structures and behaviors. Finally, the SD of a Radio Aurora Explorer mission demonstrates the proposed method.

2. A knowledge-based system design

In SD, the product can be represented by decomposed requirements and functions, which are then assigned to different designers responsible for designing detailed behaviors states, and structures to accomplish the functions, as displayed in Fig. 1. Under a knowledge perspective, SD of a complex product can be viewed as a knowledge-intensive process that a product is the embodiment of knowledge, functions are the requirements of knowledge; behaviors, states and structures are the application of knowledge; designers are the providers of knowledge. As a result, the essence of SD is integrating multiple knowledge domains from corresponding designers to achieve the desired functions.

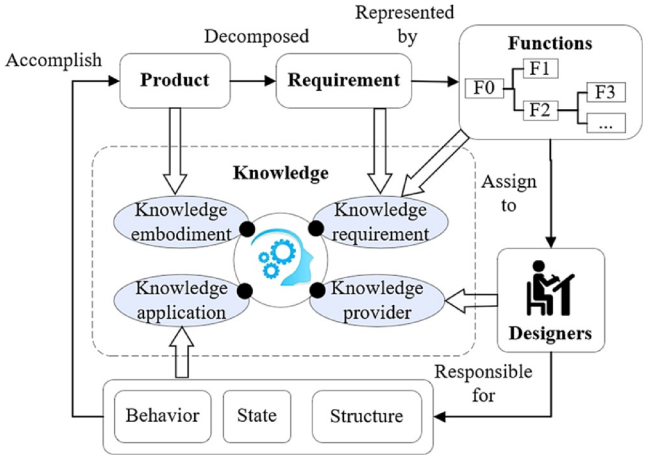


Fig. 1. A knowledge-based system design.

According to the knowledge-based SD presented in Fig. 1, functions of a product and the capability of designers should be mapped into different types of knowledge items, respectively. Achieving SD within the stipulated time relies on the matching degree between the required knowledge of decomposed functions and the level of designers' ability to master each type of knowledge domain. Decisive steps of the knowledge-based SD are functional decomposition and knowledge specification for purposes and designers.

2.1. Functional decomposition

Functions refer to the input-output relationships of materials, energy, and signals in the product. A function is usually expressed by "verb + noun", such as "determine the attitude", "store electrical energy", etc. A function tree diagram can be used to develop a graphical representation of the functional decomposition [26]. The decomposition starts from the most abstract function, and by answering "how to implement this function?", an abstract function expands into multiple sub-functions to form a "tree".

There are many approaches to build a function tree diagram. One of the most used methods is the Function Analysis System Technique (FAST) proposed by Bytheway [27]. Through FAST, Fig. 2 shows a function tree diagram with 36 functions of SD of a Radio Aurora Explorer (RAX) mission. According to FAST's functional decomposition criteria, when a function is simple enough to map to existing or commercially available components or subsystems, the decomposition can stop. For example, "Function 3-Convert solar energy into electricity" in Fig. 2 is at the end level as a solar wing can accomplish it. The benefit of utilizing a function tree diagram is that a function is independent of the product structure to represent the essence of a product before the embodiment of a design concept is achieved, which allows one to get an estimate of the design effort earlier in SD.

2.2. Mapping functions to knowledge space

In knowledge-based SD, knowledge is the link between product functions and the design team. The decomposed functions and their integration are achieved through the expertise of designers. Zhang defines technical knowledge as the professional skills used by designers in design activities that can be acquired through professional learning or actual product development. Technical knowledge is divided into general knowledge (GK) and product knowledge (PK) [29].

(1) GK is defined as the skills in a particular domain, which can be acquired through formal education, structured study, or systematic training. It determines the capability of a designer to be component at certain functions.
(2) PK is defined as the comprehension of the specific product to be designed, which is mostly developed from prior experience in similar projects. Designers with good PK are aware of interdependent activities besides their tasks and can accomplish integration from functions to the parent function.

Accordingly, a function is developed by GK, and sub-functions are integrated by PK. We use a scale to map a function into knowledge space to evaluate the required knowledge items in designing a specific function. Based on the study by Zhang and Thomson [26], we use knowledge magnitude to indicate the importance of a certain knowledge item to the function development, where a higher value indicates a severer requirement. All knowledge items should be limited to the same scale range, expressed by $[0, r]$ $(r \in N+)$. The most common situation is that $r = 2$, where a higher value typically means less uncertainty and higher knowledge requirement; on the contrary, if the magnitude of a knowledge item is 0, it is not important for the development of the function. Correspondingly, a designer's capability for

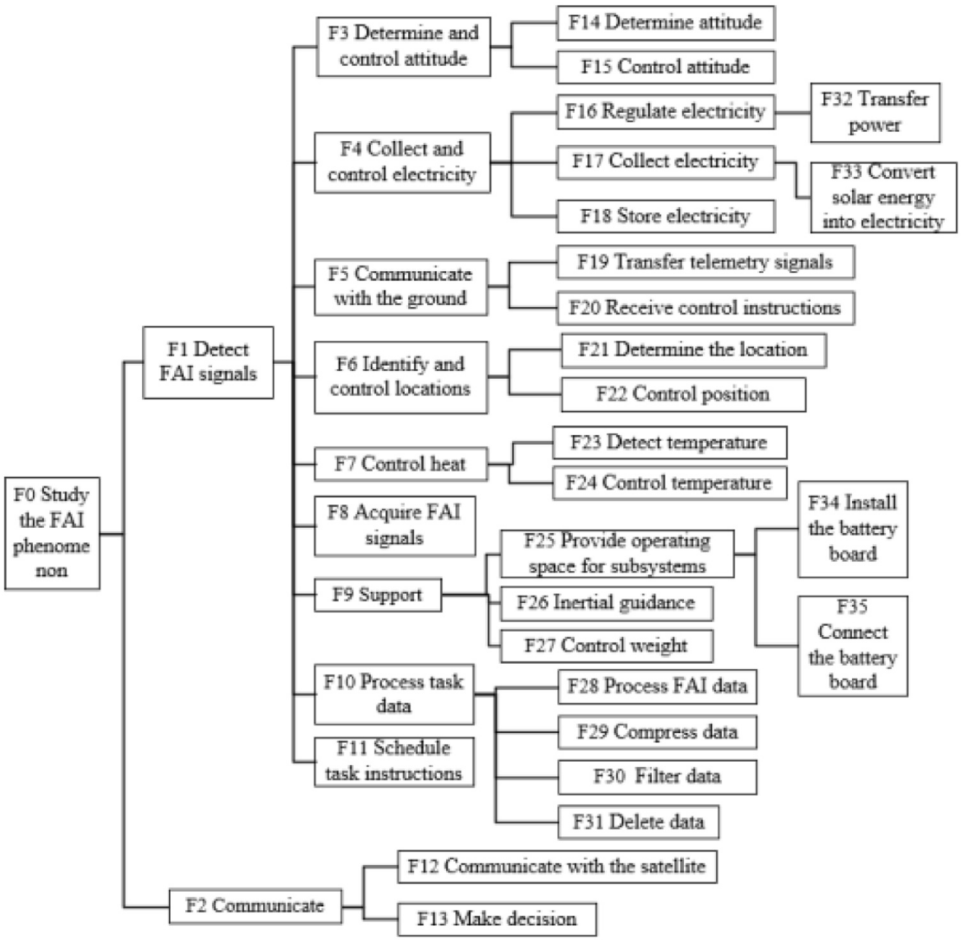


Fig. 2. Functional decomposition model of a RAX system design [28].

Table 1
Knowledge magnitude for 10 knowledge types [28].

#	Knowledge item	Magnitude		
		Not Important	Important	Very important
K1	Software engineering	0	1	2
K2	Heat transfer	0	1	2
K3	Electric heat generation	0	1	2
K4	Control	0	1	2
K5	Mechanical Engineering	0	1	2
K6	Sensor technology	0	1	2
K7	Physics	0	1	2
K8	Electrical engineering	0	1	2
K9	Management	0	1	2
K10	Communication	0	1	2

applying GK is defined with the magnitude: novice (numerical value = 0), ordinary (numerical value = 1) and expert (numerical value = 2). Table 1 shows the ten types of knowledge and their magnitudes required for the SD of the RAX mission.

If the development of a function involves n knowledge items, they can form an n-dimensional space, called knowledge vector $\boldsymbol{F}=(k_1, k_2,\ldots k_i\ldots, k_n)$, where k_i is the magnitude value of knowledge item i. Fig. 3 shows the mapping from functions of the RAX mission into knowledge vectors. Table 2 lists the knowledge vector for each of the 36 functions for the SD (Fig. 1).

3. System design digital twin framework

The knowledge-based SD in Section 2 establishes a theoretical basis

#	Function	Required knowledge									
		K1	K2	K3	K4	K5	K6	K7	K8	K9	K10
0	F0	1	1	1	1	1	1	1	1	0	0
1	F1	1	0	0	1	1	1	1	1	0	0
2	F2	1	1	1	1	1	1	0	1	0	1
3	F3	0	0	0	1	0	1	1	0	0	0

$\overrightarrow{FR}$ = (K1, K2, K3, K4, … , K10)

Map to

Fig. 3. Mapping from functions to knowledge vectors [28].

for the operation of a system design digital twin. According to the research from Tao, a complete digital twin framework should include five components: the physical entity in physical space, the virtual entity in virtual space, services for analyzing the entity, digital twin data from the entity, and connections between them [30]. Following the same structure, for a knowledge-based SD, the corresponding digital twin framework, named System Design Digital Twin in 5 Dimensions ($SDDT_{5D}$), is consists of five similar components, as illustrated in Fig. 4.

3.1. System design in physical space (SD_{PS})

SD_{PS} is the actual SD process of a product itself. In physical space, decomposed functions of the specific design object are assigned to designers. Designers with technical knowledge can develop the assigned function through sequential design tasks in a design activity. Therefore,

Table 2
List of knowledge vectors for the SD of a RAX mission [28].

#	Knowledge vectors									
	K1	K2	K3	K4	K5	K6	K7	K8	K9	K10
F0	1	1	1	1	1	1	1	1	0	0
F1	1	0	0	1	1	1	1	1	0	0
F2	1	1	1	1	1	1	0	1	0	1
F3	0	0	0	1	0	1	1	0	0	0
F4	0	0	0	0	1	1	0	1	0	0
F5	1	0	0	0	0	1	1	1	0	1
F6	0	0	0	0	0	1	1	1	0	0
F7	1	1	1	1	1	1	0	0	0	0
F8	1	0	0	0	1	1	0	0	0	0
F9	0	0	0	0	1	0	1	0	0	0
F10	1	0	0	1	0	0	0	0	0	0
F11	1	0	0	1	0	0	0	0	1	0
F12	1	0	0	0	0	1	1	1	0	1
F13	1	0	0	1	0	0	0	0	1	0
F14	1	0	0	1	1	1	0	1	0	0
F15	1	0	0	1	1	1	0	1	0	0
F16	0	0	1	0	0	0	0	1	0	0
F17	0	0	0	1	1	0	0	1	0	0
F18	0	0	0	0	1	0	1	1	0	0
F19	1	0	0	0	0	0	0	1	0	1
F20	1	0	0	0	0	0	0	1	0	0
F21	1	0	0	1	0	0	0	0	0	0
F22	1	0	0	1	1	0	0	0	0	0
F23	0	1	1	0	1	1	0	0	0	0
F24	1	1	0	1	0	1	0	0	0	0
F25	0	0	0	0	1	0	0	1	0	0
F26	0	0	0	1	1	1	0	1	0	0
F27	1	0	0	1	0	1	0	0	0	0
F28	1	0	0	0	0	0	0	0	0	0
F29	1	0	0	0	0	0	0	0	0	0
F30	1	0	0	0	0	0	0	0	0	0
F31	1	0	0	0	0	0	0	0	0	0
F32	0	0	1	1	0	0	0	1	0	0
F33	0	1	0	1	1	1	0	1	0	0
F34	0	0	0	0	1	0	0	1	0	0
F35	0	0	0	0	1	0	0	1	0	0

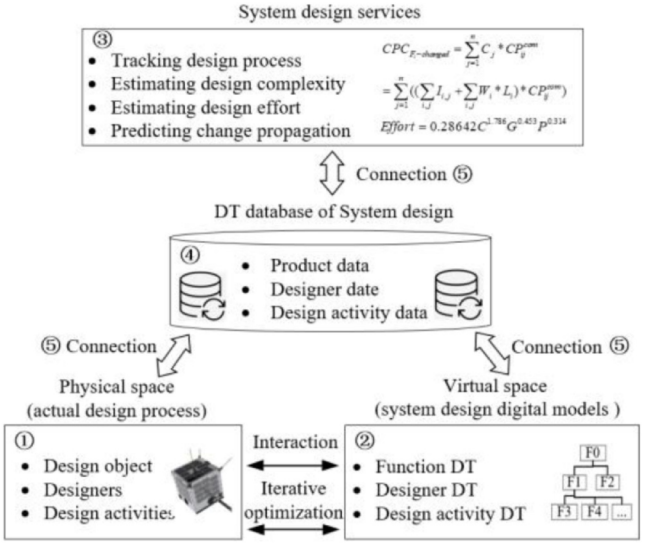


Fig. 4. System design digital twin framework [28].

main components of SD_{PS} are the design object, designers, and design activities.

3.2. System design digital model in virtual space ($SDDM_{VS}$)

As digital twin (DT) is defined as a real mapping of an entity in real word in the product life cycle, an $SDDM_{VS}$ is a digital twin of SD_{PS},

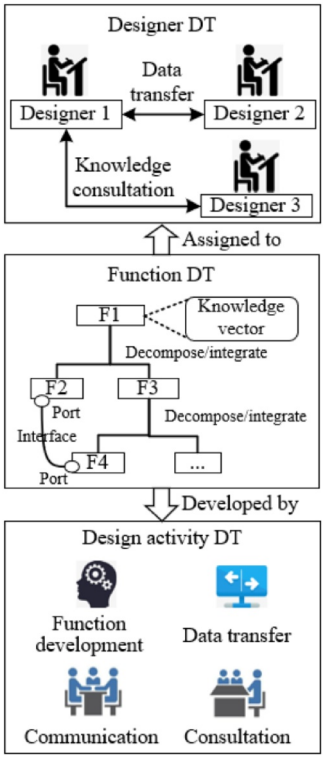


Fig. 5. Three types of digital models of system design.

which mirrors the life of an actual SD process and its characteristics. Therefore, an $SDDM_{VS}$ includes digital models of the design object (i.e., the function DT), designers (i.e., the designer DT), and design activities (i.e., the design activity DT).

The function DT represents the required knowledge; the designer DT reflects the capability of designers for possessing specific knowledge; the design activity DT connects designers and functions by knowledge application activities, such as technical development and information transfer, and consultation, etc. Fig. 5 shows the composition of the function DT, designer DT, and design activity DT and their relationships, which are illustrated in detail in [28].

3.3. System design services (SDS)

SDS is the core enabler in $SDDT_{5D}$, which can provide intelligent execution, precise estimation, and reliable control for the SD. The services are necessarily a series of mathematical models that integrate tracking, evaluation, optimization, and estimate algorithms. Analysis approaches in virtual space can be performed by specific services using real data from physical space. As illustrated in Table 3, three kinds of services are discussed in detail in Section 4.

3.4. Digital twin database (DTD)

DTD acts as the driving force of $SDDT_{5D}$, including data from physical entities, virtual models, and services, and the data regularly update with changes in real-time status. Because both SD_{PS} and $SDDM_{VS}$ come down to the design object (i.e., the product), designers, and design activities, data in DTD can be classified into the product data, the designer data, and the design activity data, as listed in Table 4.

The role of DTD has two aspects. It provides data for early-stage predetermination of design states, such as the complexity measurement of the design, the estimation of required design effort, and the prediction of the change propagation. For another, it provides data for real-time tracking of design states, such as completed design tasks, designer workload, and rest effort of the design.

Table 3
SDS services and their brief introductions.

SDS name	Introduction
Design complexity measurement	A knowledge-based complexity metric is used to measure the system design complexity.
Design effort estimation	The effort for technical development, information transfer, consolation on knowledge, and design change propagation are considered.
Design change propagation prediction	Change propagation paths in the function tree are identified, and change propagation probabilities are predicted.

3.5. Connection (CN)

CN plays as an intermedium for connecting the above four components so that the digital twin can carry out real-time and efficient information transmission, and thereby ensuring the interaction and iterative optimization between the physical space and virtual space.

$$SDDT_{5D} = \{SD_{PS}, SDDM_{VS}, SDS, DTD, CN\} \quad (1)$$

In summary, Formula (1) expresses $SDDT_{5D}$. SD_{PS} and CN are out of the scope of the research as SD_{PS} is an objective reality, and CN mostly relies on the quality of the hardware, so the focus of this article is $SDDM_{VS}$, SDS, DTD, and the unified modeling approach for them [28].

System design in the framework (Fig. 4) is an iterative optimization process. The system design digital model represents information of design objects, designers, and design activities from the design team. With the development of the system, the digital model information becomes complete. Moreover, through system services, the actual system design process can be optimized. For example, managers can more accurately identify the knowledge requirement of functions and designate more capable designers on crucial functions. Details of System Design Digital Model in Virtual Space, Designer DT, Design activity DT, System Design Services and their relationships are illustrated in [28].

4. Unified modeling by SysML

SysML is used for modeling the unified model of $SDDM_{VS}$, SDS, and DTD, because it has been designed to enable extensions that support specialized concepts by its extension mechanism, called stereotyping [8]. Stereotypes are organized into a profile. SysML itself is defined as a UML profile.

Stereotypes are used for customizing core concepts of System Design Digital Twin in 5 Dimensions ($SDDT_{5D}$). A $SDDT_{5D}$ modeling profile is constructed, as displayed in Fig. 6, where mappings and decomposition relationships, etc. constitute the relationship profile; the function DT, designer DT, and design activity DT profiles form the $SDDM_{VS}$ profile. The product data, designer data, and design activity data profiles constitute the DTD profile. The design complexity measurement, effort estimation, and change propagation prediction profiles constitute the SDS profile.

Fig. 7 shows an $SDDT_{5D}$ modeling profile that is defined by importing a SysML model that references a UML profile (i.e., UML4SysML) to reuse their classes such as «block», «property», and relationships such as «dependency».

4.1. Relationship profile

The definition of a relationship profile is described in Fig. 8. Two kinds of mapping relationships: (a) mapping at the same level and (b) mapping between two different levels, are named as H mapping and V mapping, respectively, by extending the «dependency» relationship.

Subclasses of the two mappings are defined in Table 5, in which the H mapping includes «peer to», «interact with», «require», «assigned to», «possess» and «propagate to». The V mapping includes «decomposed to», «integrate to» and «consult».

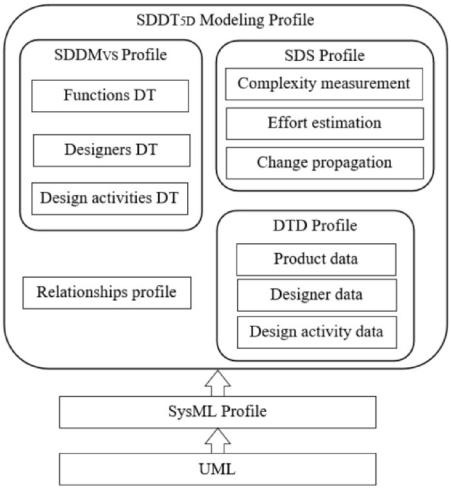


Fig. 6. $SDDT_{5D}$ modeling profile.

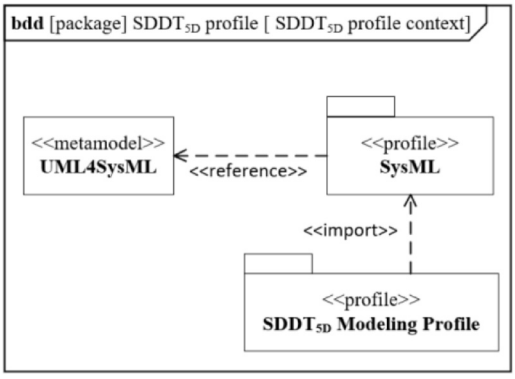


Fig. 7. Definition of $PDDT_{5D}$ modeling profile.

4.2. $SDDM_{VS}$ profile

The $SDDM_{VS}$ profile includes definitions of functions, interfaces, designers, and design activities. As illustrated in Fig. 9, a function

Table 4
Data type in DTD.

Data type	Content
Product data	This refers to the functional requirements and their relationships, such as the decomposition, interaction, and integration of functions, which reflects the complexity of the design process.
Designer data	This refers to the design capabilities possessed by designers, which can be quantified by knowledge vectors.
Design activity data	This includes data from technical development, consultation between designers, and data exchange between functions.

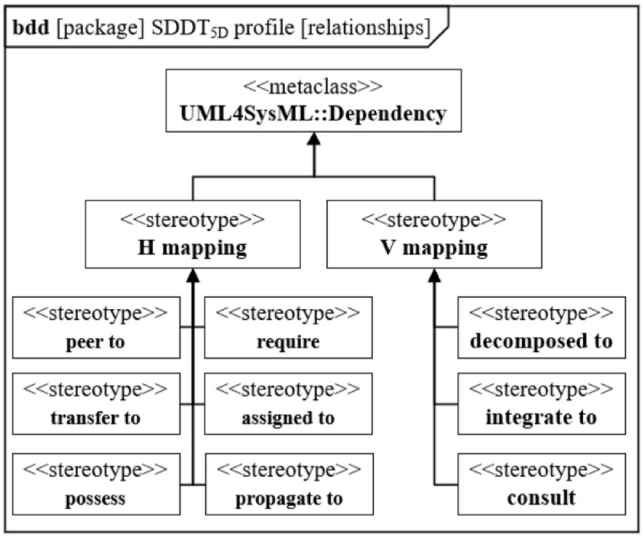


Fig. 8. Definition of the relationship profile.

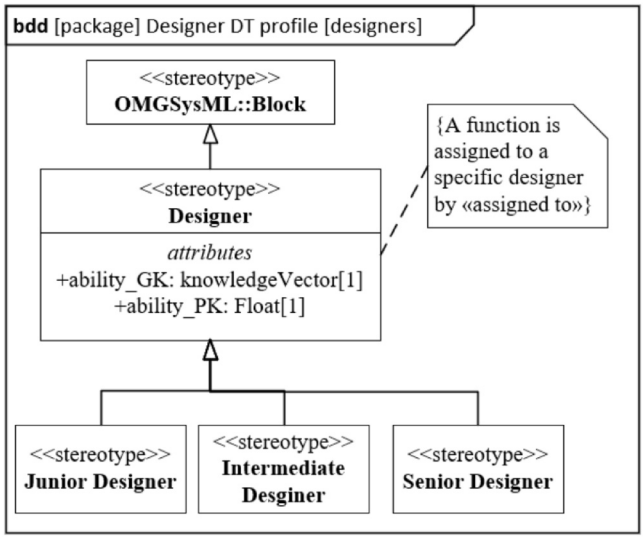


Fig. 10. Definition of the designer DT profile.

Table 5
Subclasses of the two mappings.

Name/Type	Introduction
peer to/H	Sub-functions have the same parent function
require/H	A function requires certain knowledge items
transfer to/H	A function transfers information to another
assigned to/H	A function is assigned to a specific designer
possess/H	A designer possesses technical knowledge
propagate to/H	A change propagates from a function to another
decomposed to/V	A function is decomposed to sub-functions
integrate to/V	Sub-functions are integrated to the parent function
consult/V	A designer consults another on knowledge

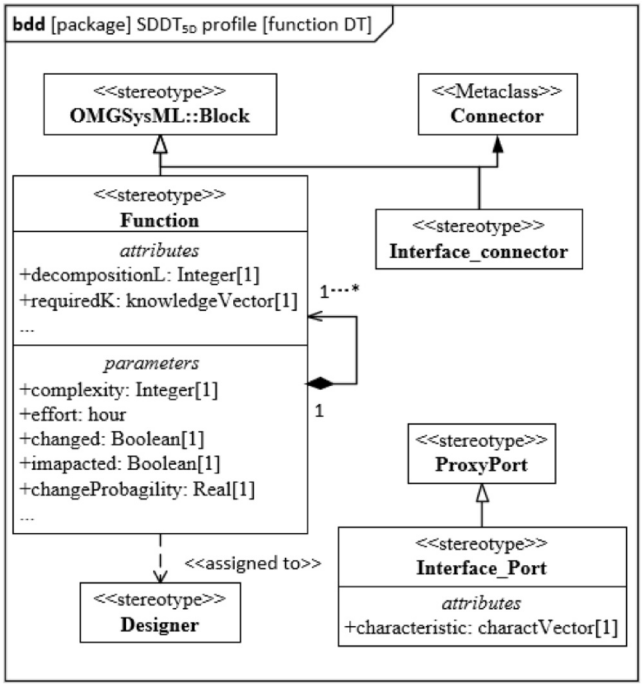


Fig. 9. Definition of the function DT profile.

stereotype is generated from a Block. It contains attributes and parameters, such as the decomposition level, required knowledge vector, development complexity, consumed effort, and change probability. A function can be a child function, a parent function, or a peer function; their relationships are specified by «decomposed to», «integrate to» and «peer to». Additionally, an interface consists of a connector (named Interface_connector) and two ports (named Interface_Port). An Interface_connector extends from the Connector and inherits from the Block; an Interface_Port inherits from the ProxyPort.

A designer stereotype is defined by inheriting from the Block. Two attributes of a designer stereotype are provided, the ability_GK represents the designer's ability of general knowledge, and the ability_PK refers to the designer's ability of product knowledge. Moreover, a designer is classified into a junior designer, an intermediate designer, or a senior designer based on the ability_GK and ability_PK, as illustrated in Fig. 10.

In Fig. 11, a design activity stereotype is defined by extending a use case, activity and interaction, and can be classified into function development, information transfer, and knowledge consulting. Furthermore, values and parameters are defined, such as the effort, interface

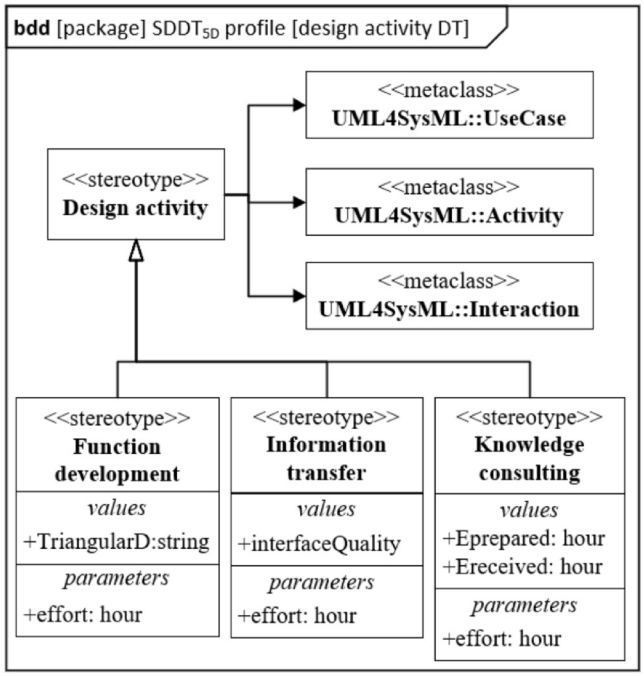


Fig. 11. Definition of the design activity DT profile.

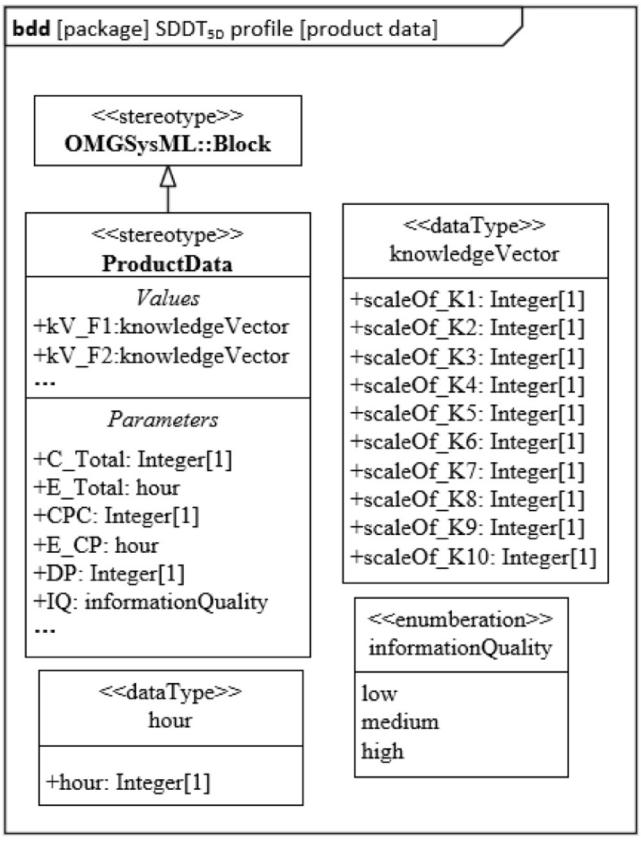


Fig. 12. Definition of product data.

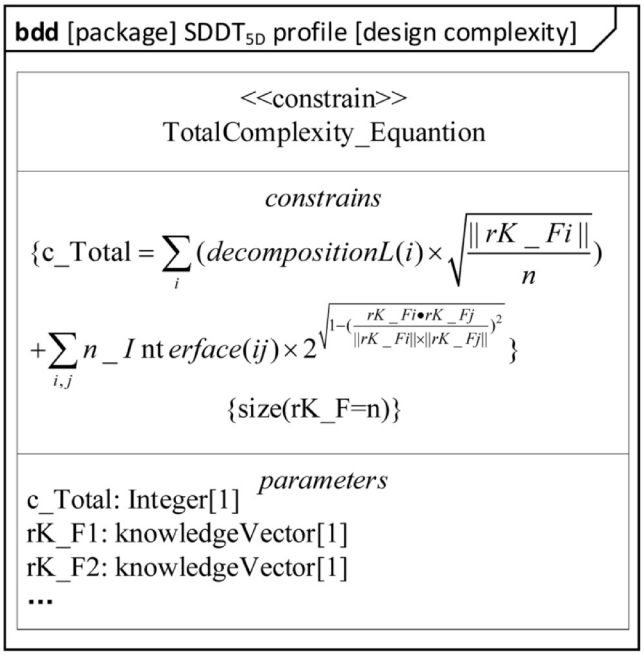


Fig. 13. Definition of design complexity constrain.

quality, and preparation effort.

4.3. Digital twin database profile

A DTD profile includes the product data stereotype, designer data stereotype, and design activity data stereotype. These are defined by inheriting from the Block, specifying values and parameters, and

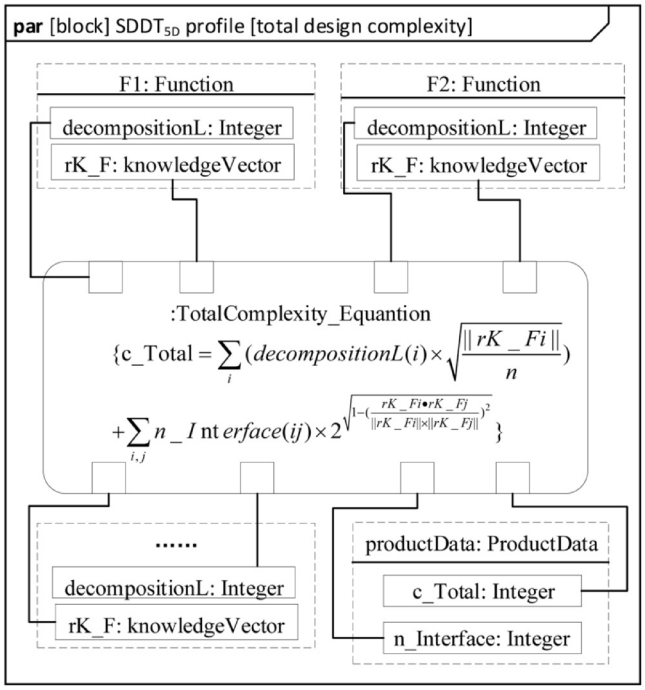


Fig. 14. Parameter diagram of total design complexity.

Table 6
The qualitative and quantitative aspects of initial data [28].

Data Type	Content	Values
Product	Functional decomposition	Fig. 2
	Interfaces	Fig. 11 in [28]
	Knowledge vectors	Table 2
	NoD	1
	IQ	Medium (2)
	Changed function	F17
Designers	AFD	2:5:3
	AFI	50%
Design activities	TriangularD	(40,64,50)
	$E_{prepared}$	2
	$E_{receive}$	2
	InterfaceQ	2

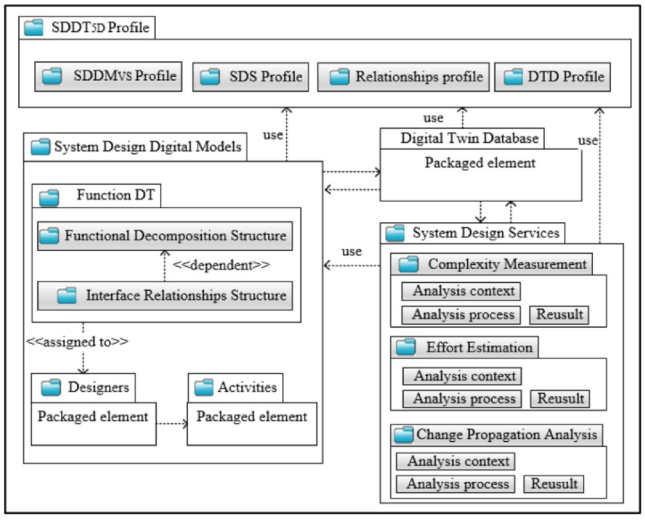


Fig. 15. The model organization.

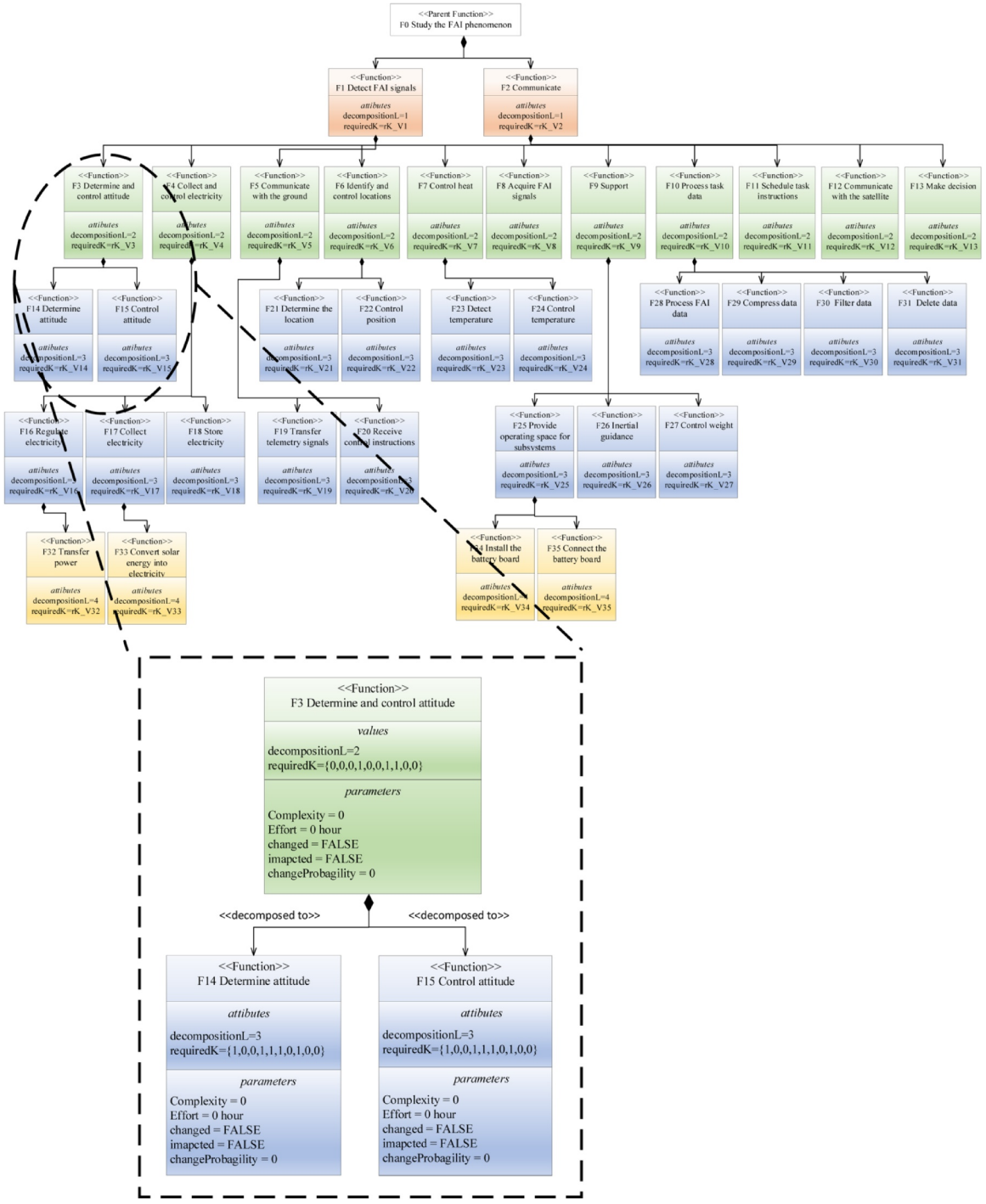


Fig. 16. Functional decomposition of the RAX CubeSate system design.

creating new data types. For example, in Fig. 12, ProductData is a kind of Block, and it has attributes like required knowledge vectors mapped from functions, total complexity (C_Total), total effort (E_Total), number of designers (NoD) and information quality (IQ), etc. In a similar approach, DesignerData, like the ability of function development (AFD) and functional integration (AFI) and the DesignActivityData, can be defined.

4.4. System design services profile

SysML supports different types of engineering analyses using parametric models, which capture constraints on the properties of a system and can then be evaluated by an appropriate analysis tool. Fig. 13 shows the constraint block of total design complexity calculated by Eq. (6). Fig. 14 illustrates a parametric diagram for the constraint block of overall design complexity. The parameters of the constraint properties rK_F and decompositionL are bound the corresponding parameters of the block Function, the constraint properties c_Total and n_Interface are bound the parameters of block Product Data. Equations related to design effort estimation and design change propagation prediction can be modeled in the same way.

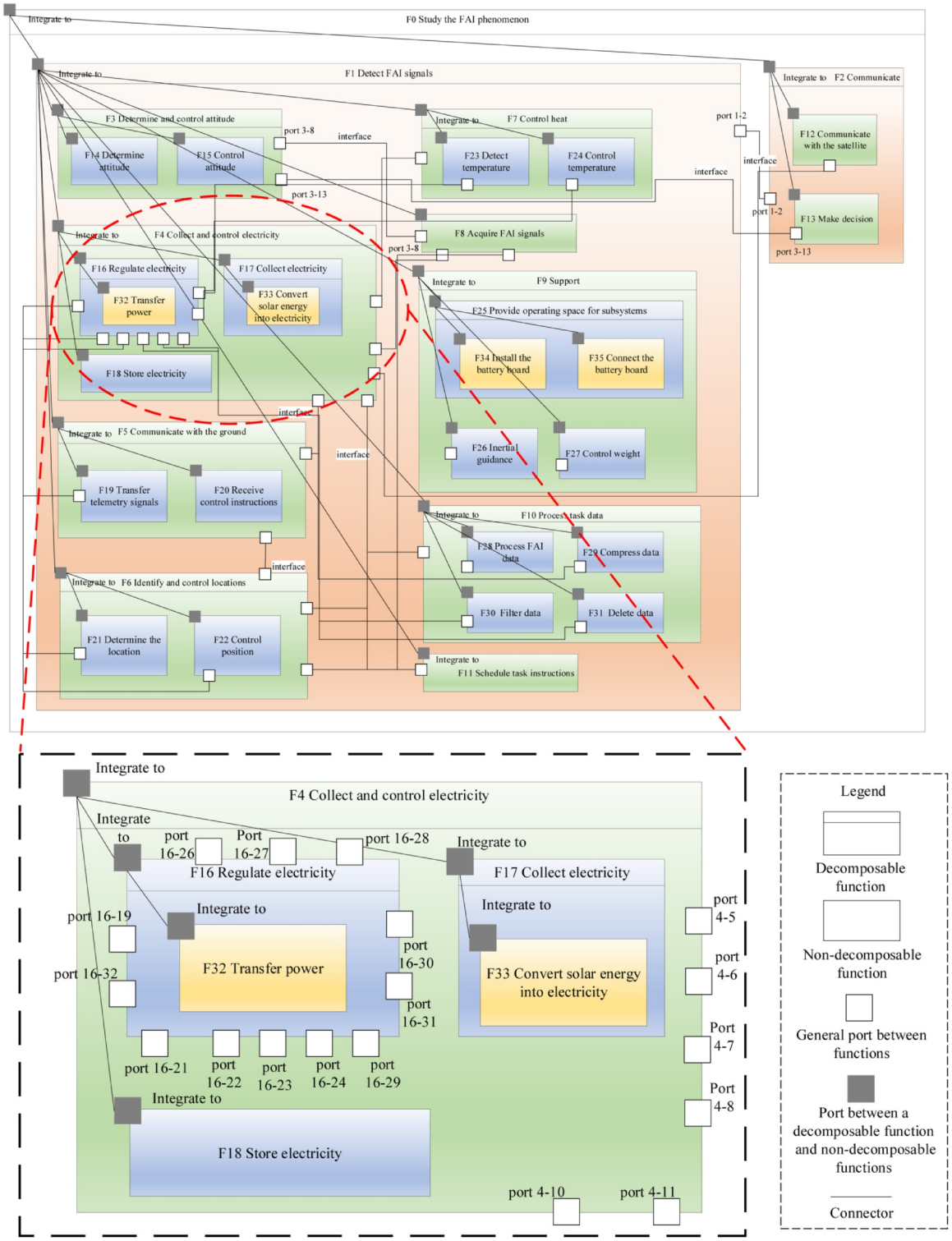


Fig. 17. Interface relationships among functions.

5. Case study and discussion

System design of space mission is challenging as functions are coupled and constrained by the limited available commodities. Furthermore, additional complexities arise because designers describe the system in a document-based way. Radio Aurora Explorer (RAX) is a standard mission in the aerospace field. The primary objective of the mission is to study the formation of magnetic field-aligned plasma irregularities (FAI). Based on a RAX CubeSat project, in 2012, the SSWG (Space Systems Working Group) of INCOSE demonstrated the feasibility of the MBSD approach in reality, where SysML could formally specify every aspect of a system [31]. Accordingly, in the following section, the modeling approach proposed in this paper is applied to the system design of a virtual RAX CubeSat mission.

Necessary information has been given, for example, the decomposed functions are revealed in Fig. 2, the required knowledge is provided in Table 2, and the interfaces among functions are shown in Fig. 11 in [28]. Besides, the quantitative aspects of designers and design activities

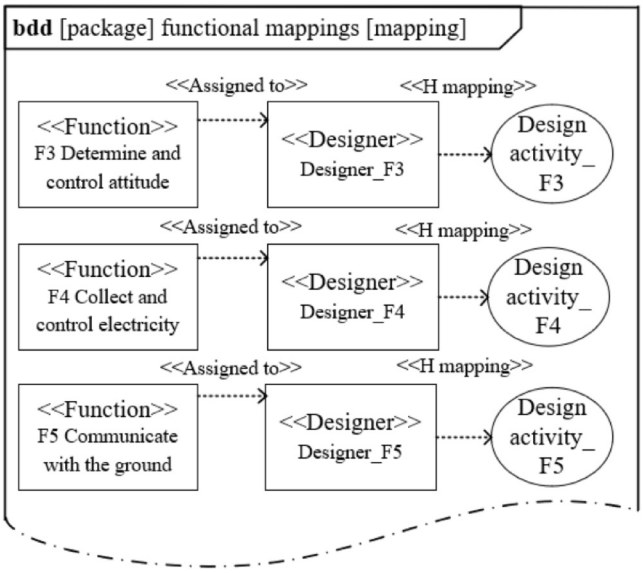


Fig. 18. Functions are assigned to designers and activities.

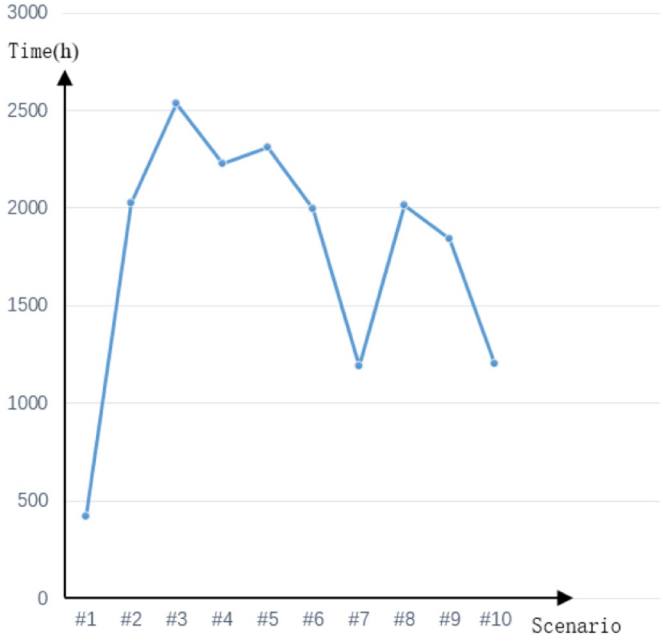


Fig. 19. Simulation results of the 10 scenarios.

Table 7
Output data [28].

Type	Name	Value
Complexity	C_Total	195
Effort	E_Total	4819
	E_{F16}	303
	$E_{Transfer\text{-}F16}$	60
	$E_{Consult\text{-}F16}$	29
Change propagation	$E_{CP\text{-}F17}$	419
	Table 8	

Table 8
Change propagation analysis of F_{17} [28].

Changed	Impacted	Change probability
F17	F33	51%
	F18	43%
	F4	88%
	F1	71%

are required, which are shown in Table 6. To model the amount of effort needed, we make the following assumptions: the AFD value in the designer data based on the ratio of junior, intermediate, and senior designers is 2: 5: 3. Each designer is assigned a function development task, so the NoD value is 1 for 36. *TriangularD* (40, 64, 50) is selected as the triangle distribution in the design activity data, and the information receiving and preparation time are set to 2 h, respectively. Furthermore, function 16 (F_{16}) is the key function because it has the most interfaces, and function 17 (F_{17}) is selected as the changed function.

The first step is to specify models and organize them initially. An overall package structure based on the $SDDT_{5D}$ is used to establish the model organization, as displayed in Fig. 15. Thus, different models are organized into specific packages according to the aim of them.

Details of each package are modeled. For example, decomposition of the functions is described by the SysML Block Definition Diagram (BDD) because functions are treated as components that can be "assembled", as illustrated in Fig. 16. In the BDD, "children, parent, and peer" relationships among them are represented by the profile of the relationship. For the RAX CubeSat, there are 36 functions, and the top function is "study the FAI phenomenon", which is on 0 decomposition level.

Interfaces of the functions are then described by the SysML Internal Block Diagram (IBD), as exhibited in Fig. 17.

One function is assigned to one designer and different design activities, as illustrated in Fig. 18. Then, design activities are elaborated to show the behavior of every designer by the SysML Sequence Diagram or Activity Diagram using the design activity DT profile.

Based on the assumption and through the system services modeled by the parametric models, values of parameters in equations on design complexity assessment, workload estimation, and function change analysis are specified by linking to corresponding data. Finally, properties of the system design, such as design complexity and effort, can be evaluated by appropriate analysis tools like MATLAB. The output data of the analysis is listed in Table 7.

Table 9
Scenarios for simulating the change of RAX system design.

Scenario	Changed functions	Changed interfaces	Changed knowledge
#1	F17 + Impacted functions	No change	Mechanical
#2	F17 + Impacted functions	No change	All
#3	Two new functions are added,F23, F31	Basic parent-child interfaces	All
#4	Two new functions are added,F23, F31	One extra interface, F25-F34	All
#5	Two new functions are added,F23, F31	Two extra interfaces, F25-F34, F25-F35	All
#6	Two new functions are added,F23, F31	Basic parent-child interfaces	Mechanical
#7	Two new functions are added,F23, F31	One extra interface, F25-F34	Mechanical
#8	Two new functions are added,F23, F31	Two extra interfaces, F25-F34, F25-F35	Mechanical
#9	F17 + Impacted functions	No change	Mechanical and control
#10	Two new functions are added,F23, F31	One extra interface, F25-F34	Mechanical and control

It can be seen from Table 7 that the total complexity (C_Total) is 195, and the total effort (E_Total) is approximately 4819 h. Based on the estimated design effort, managers can use the trade-off analysis by setting SysML models' different values to optimize the design progress. For example, based on the result, managers can more accurately identify the knowledge requirement of functions and designate more capable designers on key functions.

To assess the proposed method's practical usability, two students (Group A) and two experienced managers (Group B) were asked to participate in an experiment about estimating the total effort of the RAX CubeSate system design. The two students having the SysML knowledge background used the proposed unified models to represnt the system design, while the two managers in Group B used a traditional document-based approach, such as the requirements specification by Microsoft Word and the interface definition document by Microsoft Excel, to specify the system design. The student group spent about 48 h in modeling the system model by the $SDDT_{5D}$ modeling profile and spend 4 more hours in getting the correct total effort. Compared with students, managers in Group B spend less time in specifying the systems, about 24 h. However, the two managers used more time, about 10 h, in getting the correct total effort as they wasted more time in understanding the ambiguity issues. Although the students spent more time in modeling the system model, they saved more time in analyzing the design complexity. Moreover, the unified model can be reused in a more detailed stage to avoid the ambiguity between the understandings of different designers.

Additionally, the effort required to develop the key function F_{16} is about 303 h, accounting for 6.29% of the total effort, which can prove that this function is one of the core functions. The information transfer effort ($E_{tansfer\text{-}F16}$) required during the development of F_{16} is 60 h, and the knowledge consultation effort of it ($E_{Consult\text{-}F16}$) is about 29 h.

Table 8 presents the impact of the change propagation of F_{17}. The most affected functions are F_{18}—store electricity (88%), F_{15}—control attitude (81%), and F_{21}—determined the location (81%). According to the prediction, the manager can decide whether to change the corresponding design parameters. Moreover, in Table 8, the additional effort for changing F_{17} ($E_{CP\text{-}F17}$) is approximately 419 h, which significantly delays the design cycle.

A significant function of the established unified model is that it can simulate different scenarios by conveniently changing different parameters in the SysML parameter diagram, thereby helping the designer to determine critical functions and interfaces in the early design stage. For example, Table 9 lists ten different scenarios for simulating the change of RAX system design. The simulation case includes all 36 functions, ten kinds of knowledge, and multiple interfaces of all RAX system designs, assuming that function 17 has been changed. Among them, scenario 1 is the basic scenario; for scenario 2, only the affected functions are simulated; then, two new random functions are added to the list of changed functions (scenario 3); then, with different knowledge vectors 1 and 2 random interfaces for simulation (scenario 4 to 10). The primary purpose of these scenarios is to cover many possible examples of function change propagation: addition/deletion of functions, addition/removal of knowledge items, and modification of interfaces. Simulation results show that additional work is required if there are any changes in the design, as shown in Fig. 19.

However, designers criticized the intensive manual work required for building a model. In reality, designers were unwilling to spend so much time in creating SysML diagrams. More importantly, an experienced manager said that they preferred making decisions by referencing similar cases of a previous design or believing their experience. They suggested a few improvement ideas for future work, such as linking the model to available information databases, and building ontologies for common cases to reuse or reference them rather than creating new ones.

6. Conclusion and future work

Leading MBSE methodologies like Harmony SE, OOSEM, SA, etc., improve upon traditional DBSE by formalizing the application of modeling to support requirements analysis, function analysis, and design synthesis. However, one of the core issues that companies care about is how to effectively evaluate, predict, and manage the complexity of a complex project in the early SD stage. The existing methods still have the problem of inaccuracy. The main reason is that the actual SD in a physical space is inconsistent with the theoretical simulation of the virtual space. The emergence of digital twins (DT) has provided an effective way to provide intelligent, efficient, and real-time services for SD to ensure the consistency between the virtual model and the physical state. In order to integrate DT with MBSE to help project managers understand the design complexity in early stage:

(1) According to a knowledge-based SD theory, a System Design Digital Twin in 5 Dimensions ($SDDT_{5D}$) framework is introduced, which can map the actual SD process into the corresponding virtual space.
(2) The core concepts and relationships of $SDDT_{5D}$ are customized by a SysML profile that keeps every view of the digital system model unified.

Although SysML modeling tools, such as Magicdraw, enable designers to create and document a system model, many designers in China find it unacceptable to spend too much time in learning such a graphical modeling language without appropriate patterns. Besides, it is difficult for the machine to understand the semantics of the SysML diagrams. In the future, we will carry out in-depth research on:

(1) Linking available information databases to create reusable cases and appropriate pattern, and
(2) Represent systems design information to be both user-understandable and machine-readable.

Author statement

We have made substantial contributions to the conception or design of the work or the acquisition, analysis, or interpretation of data for the work named *Unified Modeling for Digital Twin of a Knowledge-Based System Design*. We have drafted the work and revised it critically for relevant intellectual content. We have approved the final version to be published. We agree to be accountable for all aspects of the work in ensuring that questions related to the accuracy or integrity of any part of the work are appropriately investigated and resolved.

All organizations that have made substantial contributions to the work reported in the manuscript, including those who provided editing and writing assistance but are not authors, are named in the manuscript's Acknowledgments section.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was funded by the National Natural Science Foundation of China (No. 51905493, and No. 51775517), the Shanghai Key Lab of Advanced Manufacturing Environment (No. KT20190603).

References

[1] A.M. Madni, et al., M. Model-based systems engineering: motivation, current status, and research opportunities, Syst. Eng. 21 (3) (2018) 172–190.

[2] T. Huldt, I. Stenius, State-of-practice survey of model-based systems engineering, Syst. Eng. 22 (2) (2019) 134–145.
[3] A.M. Madni, et al., Leveraging digital twin technology in model-based systems engineering, Systems 7 (1) (2019) 7, , https://doi.org/10.3390/systems7010007.
[4] H.P. Hoffmann, Deploying model-based systems engineering with IBM® rational® solutions for systems and software engineering, 2012 IEEE/AIAA 31st Digital Avionics Systems Conference (DASC), IEEE, 2012, pp. 1–8.
[5] D. Dori, Model-based Systems Engineering With OPM and SysML, Springer, New York, 2016, pp. 2–15.
[6] S. Friedenthal, et al., A Practical Guide to SysML: the Systems Modeling Language, Morgan Kaufmann, 2014, pp. 3–10.
[7] T. Weilkiens, SYSMOD-The Systems Modeling Toolbox-Pragmatic MBSE With SysML, Lulu.com, 2016, pp. 10–19.
[8] S. Friedenthal, et al., OMG systems modeling language (OMG SysML) tutorial, INCOSE Intl. Symp. 9 (2006) 65–67.
[9] K. Efthymiou, et al., Manufacturing systems complexity review: challenges and outlook, Procedia CIRP 3 (2012) 644–649.
[10] P. Bossaerts, C. Murawski, Computational complexity and human decision-making, Trends Cogn. Sci. (Regul. Ed.) 21 (12) (2017) 917–929.
[11] O. Kulak, Applications of axiomatic design principles: a literature review, Expert. Syst. Appl. 37 (9) (2010) 6705–6717.
[12] X. Zhang, V. Thomson, A knowledge-based measure of product complexity, Comput. Ind. Eng. 115 (2018) 80–87.
[13] J.J. Shah, G. Runger, Misuse of information-theoretic dispersion measures as design complexity metrics, ASME 2011 IDETC/CIE Conference, ASME, 2011, pp. 395–404.
[14] I. Ullah, et al., Engineering product and process design changes: a literature overview, Procedia CIRP 56 (2016) 25–33.
[15] Y. Lu, et al., Digital Twin-driven smart manufacturing: connotation, reference model, applications and research issues, Robot. Comput. Integr. Manuf. 61 (2020) 101837.
[16] K. Zhang, et al., Digital twin-based opti-state control method for a synchronized production operation system, Robot. Comput. Integr. Manuf. 63 (2020) 101892.
[17] P. Aivaliotis, et al., The use of Digital Twin for predictive maintenance in manufacturing, Int. J. Comput. Integr. Manuf. 32 (11) (2019) 1067–1080.
[18] M. Grieves, Virtually Perfect: Driving Innovative and Lean Products Through Product Lifecycle Management, Space Coast Press, Cocoa Beach, 2011, pp. 15–32.
[19] C. Qian, et al., A real-time data-driven collaborative mechanism in fixed-position assembly systems for smart manufacturing, Robot. Comput. Integr. Manuf. 61 (2020) 101841.
[20] J. Cheng, et al., DT-II: digital twin enhanced Industrial Internet reference framework towards smart manufacturing, Robot. Comput. Integr. Manuf. 62 (2020) 101881.
[21] P. Aivaliotis, et al., Methodology for enabling Digital Twin using advanced physics-based modelling in predictive maintenance, Procedia CIRP (2019) 417–422.
[22] G. Bachelor, et al., Model-based design of complex aeronautical systems through digital twin and thread concepts, IEEE Syst. J. (2019) 1–12.
[23] D. Thomas, et al., A holistic system lifecycle engineering approach–closing the loop between system architecture and digital twins, Procedia CIRP 84 (2019) 538–544.
[24] S.O. Erikstad, Design patterns for digital twin solutions in marine systems design and operations, The 7th International Conference on Computer Applications and Information Technology in the Maritime Industries, COMPIT 2018, 2018, pp. 1–15.
[25] M. Schluse, et al., Experimentable digital twins for model-based systems engineering and simulation-based development, 2017 Annual IEEE International Systems Conference (SysCon), IEEE, 2017, pp. 1–8.
[26] H.A. Bashir, V. Thomson, Estimating design effort for GE hydro projects, Comput. Ind. Eng. 46 (2) (2004) 195–204.
[27] W. Bytheway, Fast Creativity and Innovation: Rapidly Improving Processes, Product Development and Solving Complex Problems, J. Ross Publishing, 2007, pp. 6–18.
[28] H.Q. Wang, et al., Digital twin-based product design process and the design effort prediction method, Comput. Integr. Manuf. Syst. (2020) 1–15 5-25 http://kns.cnki.net/kcms/detail/11.5946.TP.20200506.1719.008.html.
[29] X. Zhang, V. Thomson, Modelling the development of complex products using a knowledge perspective, Res. Eng. Des. 30 (2) (2019) 203–226.
[30] F. Tao, et al., Five-dimension digital twin model and its ten applications, Comput. Integr. Manuf. Syst. 25 (1) (2019) 1–18.
[31] S.C. Spangelo, et al., Applying model based systems engineering (MBSE) to a standard CubeSat, 2012 IEEE Aerospace Conference, IEEE, 2012, pp. 1–20.